

FABO: Agent-Guided Discovery of Joint Breakpoint Optimization for Timing-Driven Routing Trees

Shang Liu, Wenji Fang, *Graduate Student Member, IEEE*, Jing Wang, Hongxin Kong, Yao Lu, Zhiyao Xie, *Member, IEEE*

Abstract—The topology of a routing tree determines how a multi-pin net branches and shares physical wire, directly affecting wirelength, congestion, capacitance, and delay. We study a central early-stage routing problem: minimizing wirelength while bounding the root-to-sink path stretch for every sink. SALT is the state-of-the-art constructive algorithm for this problem. We ask whether language-model-guided search can discover a constructive algorithm that improves on SALT. To make this search reliable, we develop an agent framework that combines parallel exploration with independent checking. Applied to SALT, the framework discovers a structural limitation: SALT repairs one sink path at a time and therefore never jointly decides where paths sharing root-side wire should split. This sink-local choice can split the paths too early and duplicate wire. This discovery leads to Flow-Aware Breakpoint Optimization (FABO), which jointly optimizes breakpoints across root-to-sink paths that share wire while preserving every sink’s stretch budget. Across 1.29 million ICCAD15 nets and SALT’s 20-point stretch-tolerance schedule, FABO reduces average FLUTE-normalized wirelength at every setting, with peak same- ϵ reductions of 0.83% overall and 2.66% for nets with at least 30 pins. With $1.3\times$ SALT’s runtime, FABO-FAST identifies and optimizes most nets for which FABO provides a substantial wirelength reduction. Code is available at <https://github.com/DevinShang/routing-FABO>.

Index Terms—Steiner tree, routing tree, timing-driven routing, breakpoint sharing, agent-guided algorithm discovery

I. INTRODUCTION

Before detailed routing, a routing tree decides where the root-to-sink paths of a multi-pin net branch and which paths share physical wire. These decisions directly affect total wirelength, routing demand, interconnect capacitance, congestion, and root-to-sink delay [1], [2]. The effect is strongest on high-fanout nets: one root-side segment may serve many sinks, so splitting those paths early can repeat substantial wire across the resulting branches.

FLUTE rapidly constructs near-minimum-wirelength rectilinear Steiner trees but does not bound the path detour of each sink [2], [3]. SALT [4] instead minimizes wirelength subject to a hard normalized-stretch bound for every sink. This is a fundamental early-design formulation: its geometry-only constraint applies before detailed resistance, capacitance, or sign-off timing is available, yet it protects every sink individually. SALT is the state-of-the-art constructive algorithm for this problem. The key question is whether a new algorithm can reduce wirelength beyond SALT under the same per-sink

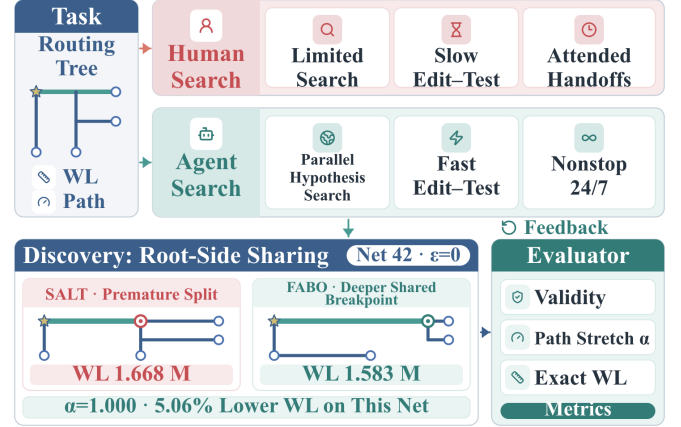


Fig. 1: **Agent-guided discovery and its routing outcome.** The upper panel contrasts the search capabilities of the human and agent workflows. The lower panel shows the recorded superblue16 net 42 case at $\epsilon = 0$: SALT and FABO both achieve $\alpha = 1.000$, while FABO moves the shared breakpoint deeper and reduces wirelength by 5.06%.

budgets, giving later physical-design stages a lighter routing structure without weakening any path guarantee.

PD-Rev balances wirelength and path detour in the Prim–Dijkstra family [5], [6]; PatLabor [7] constructs Pareto sets for wirelength and maximum path length; and Wu et al. [8] optimize Elmore delay under a wirelength requirement. These objectives are complementary, but none is a direct replacement for SALT under the per-sink normalized-stretch constraint in Section II.

From agent search to a new algorithm. Recent program-search systems show that language-model-guided search can discover new mathematical and heuristic programs [9]–[12]. Existing EDA agents mainly orchestrate existing tools [13]–[15], while algorithm-configuration methods tune parameters of a fixed procedure [16]–[18]. We instead build an agent-guided research system to develop and validate a new routing-tree construction algorithm.

A serial agent has two weaknesses in this NP-hard search. It follows one reasoning trajectory, so after adopting a construction it tends to patch that construction instead of exploring a fundamentally different one. It must also carry each idea through specification, implementation, verification, and evaluation; an unchecked error at any stage then shapes every later decision. Our framework addresses both weaknesses through multi-agent generation and checking. Eight isolated Hypothesis Agents explore different structural mechanisms, the Critic

Shang Liu, Wenji Fang, Jing Wang, Yao Lu and Zhiyao Xie are with the Department of Electronic and Computer Engineering at the Hong Kong University of Science and Technology, Hong Kong SAR, China. Corresponding Author: Zhiyao Xie (eezhiyao@ust.hk).

Hongxin Kong is with Synopsys, United States.

checks their assumptions and converts the strongest directions into fixed specifications, and three Programmer–Verifier chains implement and independently verify those specifications in parallel. The fixed Evaluator admits results from valid trees into the SSOT for the next round.

Flow-Aware Breakpoint Optimization (FABO). Per-sink budgets constrain root-to-sink path lengths; the objective minimizes total wirelength. A longer feasible path can reduce wirelength when it lets several sinks share more wire. A shared segment disappears from the tree only after every path using it is rerouted, so this saving must be evaluated jointly across those paths. SALT builds a light support tree and repairs each sink whose root path exceeds its budget: it opens a *breakpoint* on that path, keeps the wire from the breakpoint to the sink, and reconnects the breakpoint to the root through a shortest wire. The repair is greedy and per sink, and the postprocessing that follows moves one node at a time; sharing recovers only as a side effect, and the joint choice of where a group of paths should stop sharing one trunk is never represented or searched. The unexploited sharing grows with fanout, where many paths share one long root-side trunk.

In FABO, a *flow* is one root-to-sink path in the support tree. Instead of committing a flow to one repair point, FABO represents its breakpoint choices as one *feasible breakpoint interval*. The interval exactly characterizes all budget-feasible breakpoints. Moving a breakpoint deeper shortens the kept wire at least as much as it can lengthen the new root wire; therefore, the feasible breakpoints form one contiguous interval ending at the sink. Where intervals overlap, their flows can adopt one *shared breakpoint* without violating any sink budget, and FABO counts a support edge as released only when every flow using it leaves. Joint options that sink-by-sink repair never exposes thus become explicit and comparable, while every candidate tree is still checked against every sink budget.

We evaluate FABO on 1.29 million ICCAD15 nets with at least three pins under SALT’s 20-point ϵ schedule. FABO reduces average FLUTE-normalized wirelength at every setting, with peak same- ϵ reductions of 0.83% overall and 2.66% among nets with at least 30 pins. Our contributions are:

- **A multi-agent framework for reliable algorithm discovery.** Parallel Hypothesis Agents explore different construction families. At each stage, other agents check the hypotheses, specifications, implementations, and evaluation outputs before the evidence enters the next round.
- **Discovery of a structural bottleneck.** SALT repairs one sink path at a time, so it never chooses a breakpoint jointly for paths that share the same root-side wire. Its final tree can therefore split these paths earlier than necessary and duplicate wire.
- **Flow-Aware Breakpoint Optimization.** FABO changes the decision unit from one sink to the group of paths sharing root-side wire. It represents each path’s legal breakpoints as one interval and turns joint repair into selecting shared points from interval overlaps, making group-level wirelength gains explicit while preserving every sink’s path budget.

- **Quality at scale with a practical runtime option.**

Across 1.29 million nets and all 20 SALT tolerances, FABO reduces average FLUTE-normalized wirelength at every setting, with peak reductions of 0.83% overall and 2.66% for nets with at least 30 pins. With $1.3\times$ SALT’s runtime, FABO-FAST identifies and optimizes most nets for which FABO provides a substantial wirelength reduction.

II. PROBLEM FORMULATION

We adopt the rectilinear Steiner shallow-light formulation of SALT [4]. The input consists of a root pin r and a sink set $P = \{t_1, \dots, t_m\}$ on the Manhattan plane. A feasible solution is a rooted rectilinear Steiner tree T spanning the root pin r and every sink pin $t \in P$. The pin coordinates are fixed, while the routing tree T may introduce Steiner points.

Path distance and shallowness: The Manhattan distance $d_M(u, v)$ is the length of a shortest rectilinear path between points u and v . The tree-path distance $d_T(r, t)$ is the length of the unique path from the root pin r to a sink pin t in the routing tree T . The shallowness $\alpha(T)$ is the largest normalized root-to-sink path length:

$$\alpha(T) = \max_{t \in P} \frac{d_T(r, t)}{d_M(r, t)}. \quad (1)$$

Wirelength and normalization: The wirelength $WL(T)$ is the exact length of the geometric union of all rectilinear segments in the routing tree T ; coincident segments are counted once. Let T_{FLUTE} denote the FLUTE tree for the same root pin r and sink set P . Following SALT’s evaluator, we report the FLUTE-normalized wirelength

$$\beta(T) = \frac{WL(T)}{WL(T_{\text{FLUTE}})}. \quad (2)$$

Normalized delay: The experiments also report the SALT normalized-delay metric $\gamma(T)$ [4]. Let $D_T(r, t)$ be the Elmore delay from the root pin r to a sink pin t under the unit-length resistance and capacitance model. Following SALT’s evaluator, let $D_{\text{LB}}(r, t)$ denote SALT’s lower-bound delay for sink pin t , and define the per-net lower bound as $D_{\text{LB}}(r, P) = \max_{t \in P} D_{\text{LB}}(r, t)$. Then

$$\gamma(T) = \frac{\max_{t \in P} D_T(r, t)}{D_{\text{LB}}(r, P)}. \quad (3)$$

The normalized delay $\gamma(T)$ is a reporting metric rather than an optimization constraint in this work.

Stretch-constrained objective: Let the stretch tolerance $\epsilon \geq 0$ specify the allowed multiplicative detour beyond a Manhattan shortest path. For each sink pin t , define the per-sink path budget B_t by

$$B_t = (1 + \epsilon)d_M(r, t). \quad (4)$$

The rectilinear shallow-light routing problem is

$$\min_T WL(T) \quad \text{s.t.} \quad d_T(r, t) \leq B_t, \quad \forall t \in P. \quad (5)$$

Because $WL(T_{\text{FLUTE}})$ is fixed for a given net, minimizing $WL(T)$ is equivalent to minimizing $\beta(T)$. At stretch tolerance

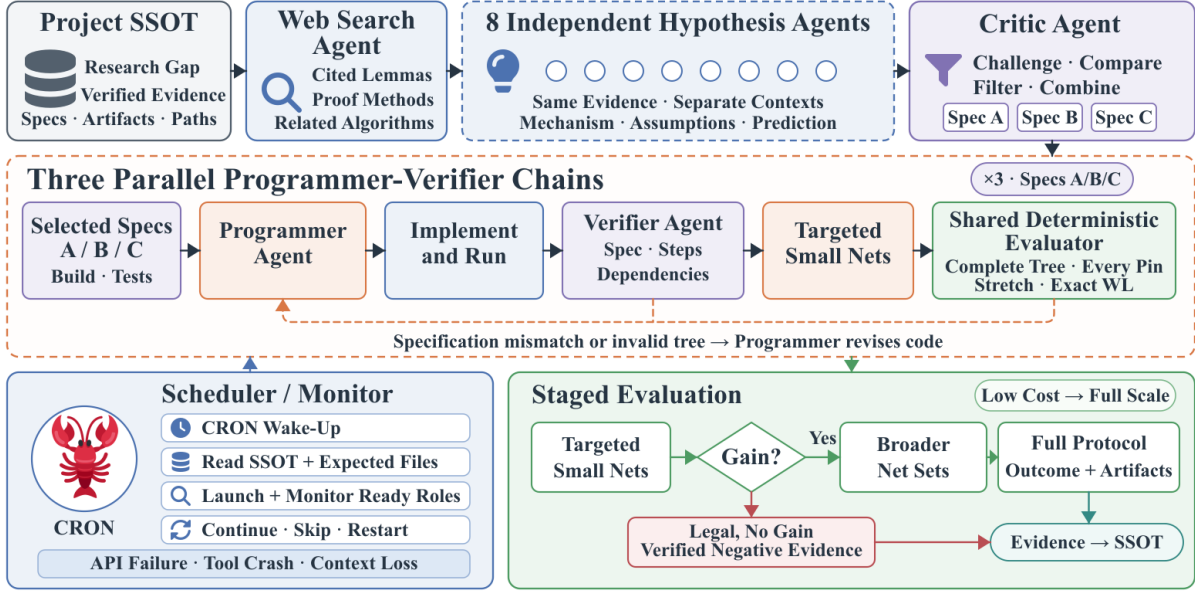


Fig. 2: **Multi-agent generation and checking in each search round.** Eight isolated Hypothesis Agents explore different structural mechanisms from the same verified evidence, and the Critic checks their assumptions before producing three fixed specs. Three Programmer–Verifier chains implement and independently check the specs in parallel. The Evaluator verifies the resulting trees, the SSOT records the evidence, and the Scheduler/Monitor keeps the file-based workflow running.

$\epsilon = 0$, every root-to-sink path must be Manhattan-shortest, giving the rectilinear minimum-arborescence limit. As the stretch tolerance ϵ increases, the path constraint relaxes and the minimum-wirelength feasible tree approaches a rectilinear Steiner minimum tree [4].

III. AGENT-GUIDED ALGORITHM-DISCOVERY FRAMEWORK

Why a serial single-agent workflow is fragile. A serial agent explores one reasoning trajectory. Once it commits to a construction, later iterations tend to refine that construction instead of proposing a fundamentally different routing structure. This path dependence can trap the search in a local optimum. Algorithm discovery also spans several dependent stages: hypothesis generation, specification, implementation, verification, and evaluation. The output of each stage becomes the input of the next. A weak hypothesis, incomplete specification, coding error, or invalid evaluation therefore distorts every later decision.

Core design: generate alternatives and check every stage. Our framework uses multi-agent collaboration throughout the workflow. At the hypothesis stage, eight agents explore different structural mechanisms and a Critic challenges their assumptions before selecting three specifications. At the execution stage, three Programmer–Verifier chains implement and independently check the specifications in parallel. A fixed Evaluator then checks complete routing trees, and the SSOT stores the verified result and its evidence before the next round. This repeated generate–check pattern broadens the structural search and improves the quality of every handoff.

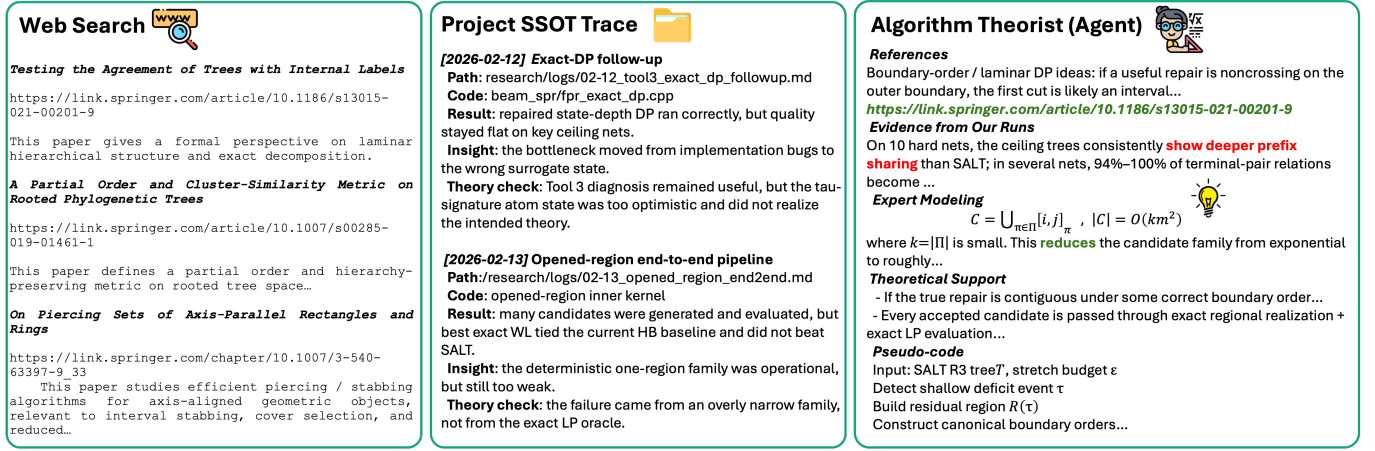
One search round. Each round starts from the single source of truth (SSOT), which records verified conclusions

and links to their code and evaluation artifacts. The Web Search Agent retrieves mathematical tools for the unresolved structural question. Eight Hypothesis Agents receive the same SSOT and references in isolated contexts and propose alternative mechanisms with explicit assumptions and measurable predictions. The Critic checks the proposals against the routing constraints and existing evidence, then converts the strongest directions into three New Idea Specs. Three Programmer–Verifier chains implement and check these specs in parallel. The fixed Evaluator verifies each complete routing tree and every path budget before measuring wirelength. The SSOT records the verified result and its evidence before the next round starts.

A. Multi-Agent Generation and Checking

Structural exploration. The SSOT defines the unresolved structural question, and the Web Search Agent retrieves relevant lemmas, proof techniques, and algorithms. Eight Hypothesis Agents independently combine this technical note with the same verified evidence. Each agent proposes a structural mechanism, states its assumptions, and gives a measurable prediction. The isolated contexts produce several search directions instead of eight refinements of one shared draft. The Critic then checks every assumption against the problem constraints and evidence, rejects inconsistent proposals, completes missing construction steps, and combines compatible mechanisms. It returns three self-contained New Idea Specs with fixed constructions and acceptance tests.

Parallel implementation and verification. Three Programmer–Verifier chains execute the specs concurrently. In each chain, the Programmer implements one spec and the Verifier independently checks the implementation,



(a) The theory reference from web-search

(b) Project SSOT trace file

(c) Analysis from algorithm theorist agent

Fig. 3: **Evidence flow through one search round.** (a) The Web Search Agent retrieves cited tools on rooted-tree structure and geometric piercing [19]–[21]. (b) The SSOT records two candidate evaluations with their code paths, measured outcomes, extracted lessons, and theory diagnoses. (c) A Hypothesis Agent combines the retrieved tools with the recorded evidence and writes a model, assumptions, theoretical support, and pseudocode. The Critic converts this material into a fixed spec for implementation and verification.

dependencies, and required tests against that spec. This check prevents implementation errors from being interpreted as algorithmic evidence. For example, in one SALT reproduction [4], empty stubs allowed the binary to run after a missing dependency disabled required postprocessing. The Verifier classified the run as an implementation failure and returned it for repair.

Evaluation and evidence update. Each verified implementation first runs on targeted small nets, then on broader net sets, and finally under the full evaluation protocol. The fixed Evaluator checks the complete output tree, verifies every root-to-sink path budget, and measures exact wirelength. Thus the framework separately establishes that the code matches the spec, the output is valid, and the valid output improves the objective. The SSOT accepts the result after these checks and supplies it to the next search round.

B. How Results Update the Next Round

Result classification. The framework assigns every run one of three labels. An *implementation failure* means that the code or its dependencies differ from the spec. A *construction failure* means that a spec-conformant implementation produces an invalid routing tree. A *verified negative result* means that the implementation matches the spec and produces a valid tree, but fails to reduce wirelength. Algorithm comparison uses evaluator measurements from valid trees.

Evidence record. Each run produces a code snapshot, build log, output tree, metric file, and verification report. The SSOT entry stores the result label, a short evidence statement, and paths to these artifacts. The next round reads this record as its starting evidence.

From failed candidates to FABO. Figure 3(b) shows two implemented candidate families that produced no gain on the targeted nets. Their diagnoses shifted the next round

from local attachment changes to the placement of shared root-side structure. Panel (c) turns this question into explicit assumptions and pseudocode. This sequence led to the flow-level breakpoint formulation in Section IV.

C. Persistent State and Recovery

File-based execution. Each agent reads its inputs from files and writes its outputs to files. An external Scheduler/Monitor polls these files at fixed intervals. It launches an agent when its inputs are ready, restarts an interrupted agent from its latest artifacts, and tracks the three execution chains separately.

IV. FABO: FLOW-AWARE BREAKPOINT OPTIMIZATION

Core idea. When a sink’s path exceeds its budget, SALT builds a new wire for that one sink and leaves every other path unchanged [4]. The total wirelength, however, is decided by how much wire the paths *share*. FABO therefore plans the new wires so that several paths can use them together: for every sink it computes every legal position for the new wire’s branch (always one contiguous interval); it lets one branch serve several sinks wherever their intervals overlap. Figure 4 shows some important definitions on a toy net.

Wirelength is paths minus sharing (Figure 4(a)). An *edge* e is one wire segment of length $\ell(e)$, and its *load* $\text{load}_T(e) \geq 1$ is the number of root-to-sink paths through it. We call each such path a *flow*: the route one sink currently uses to reach the root. An edge contributes its length to the wirelength only once, no matter how many flows it carries, so every routing tree satisfies

$$WL(T) = \sum_{t \in P} d_T(r, t) - \sum_{e \in T} (\text{load}_T(e) - 1) \ell(e). \quad (6)$$

In the example, three flows of lengths $5+7+7 = 19$ overlap on 8 units of wire, leaving $WL = 11$. The budget constraint

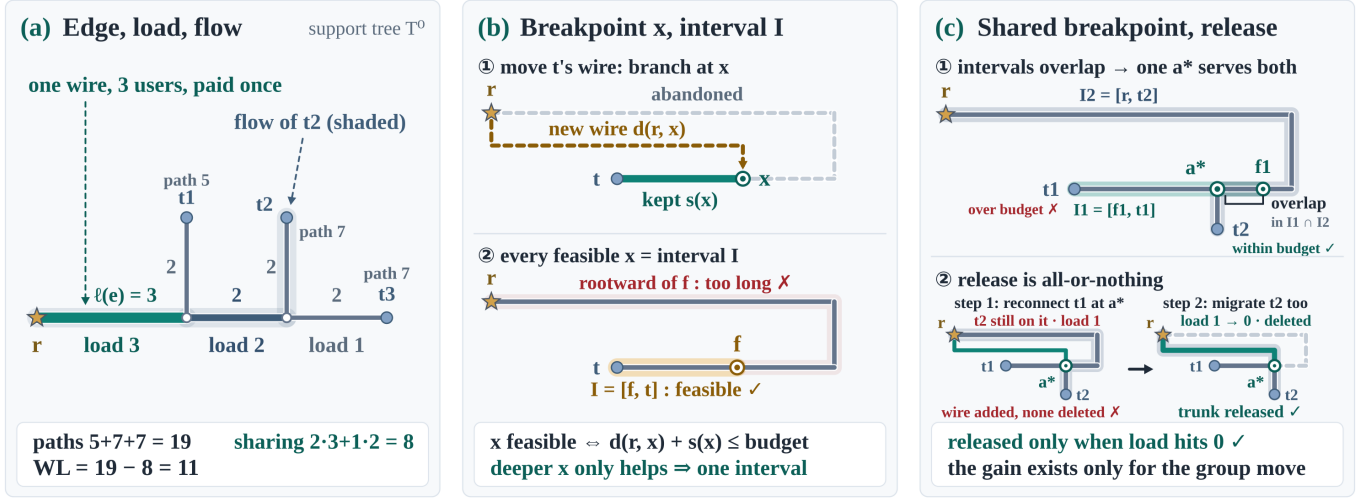


Fig. 4: (a) An edge is paid once, no matter how many flows use it: wirelength is total path length minus sharing, Eq. (6). (b) A breakpoint x keeps the wire from x to the sink and adds a new shortest wire from the root to x ; all feasible x form the interval $I = [f, t]$. (c) One point a^* in the overlap of two intervals reroutes both flows; the abandoned trunk is deleted (dashed) only in step 2, when its load reaches zero.

$d_T(r, t) \leq B_t$ from Section II only puts an upper limit on each path length in the first sum; a path may be longer than necessary, as long as it stays under its budget. FABO searches for reroutings that spend some of the available path-length budget to increase sharing and reduce wirelength: the additional sharing must save more wire than the increase in total root-to-sink path length.

Breakpoints and intervals (Figure 4(b)). When a path goes over budget, SALT connects the detected breakpoint to the root through a new shortest wire. In the running example below, that breakpoint is the sink itself, so the sink receives a private wire [4]. FABO turns this fixed repair into a choice: the new wire may stop at any point x of the flow, called a *breakpoint*—the wire from x to the sink is kept (the *suffix*, of length $s(x)$), the wire from the root to x is abandoned, and a new shortest wire of length $d_M(r, x)$ is built from the root to x . The rerouted flow is at most $d_M(r, x) + s(x)$ long, so x is *feasible* when $d_M(r, x) + s(x) \leq B_t$. Moving x toward the sink shortens the suffix by exactly the distance moved and lengthens the new wire by at most that distance, so this bound never increases, and the feasible breakpoints form one interval $I = [f, t]$ ending at the sink (Lemma 1). Its *boundary* f is the feasible point closest to the root ($f = r$ when the whole flow is within budget); for an over-budget flow, f lies where the path turns back toward the root, because only there does moving the breakpoint toward the sink shorten both the new root connection and the retained suffix.

Shared breakpoints and all-or-nothing release (Figure 4(c)). A breakpoint a^* in the overlap of I_1 and I_2 is feasible for both flows, allowing them to share one new root wire. Moving only flow 1 does not release the old trunk because flow 2 still uses it; the trunk is removed only after flow 2 also moves. FABO therefore counts an edge as saved only when all of its users leave—the all-or-nothing, per-edge account that makes the method *flow-aware*.

From these definitions to the algorithm. FABO computes every flow's interval (Section IV-B); selects a small set of breakpoints so that every interval contains one, sharing wherever intervals overlap (Section IV-C); proposes deeper breakpoints ranked by the wire their group releases (Section IV-D); and accepts a candidate only after the complete realized tree passes every budget and improves the incumbent (Section IV-E). The joint decision SALT never represents—where a group of flows should stop sharing one trunk—thereby becomes an explicit decision that FABO searches directly.

Running example. Figure 5 traces one recorded net—superblue18 net 323, eight pins, $\epsilon = 0.1125$ —through both algorithms: the top strip shows the violating support tree, SALT steps 1–2 show the private repair and local cleanup, and FABO steps 1–3 show boundary selection, joint rewiring, and final budget validation.

A. From Sink-Local Repair to Shared Breakpoints

SALT's decision. SALT traverses a light support tree and, whenever one sink path exceeds its budget B_t , connects the point where the violation is detected—its *breakpoint*—to the root through a new shortest wire; the wire from the breakpoint to the sink is kept, and fixed postprocessing passes then clean the complete tree [4]. Every choice in this loop is triggered by, and made for, one sink.

SALT's two-step repair (Figure 5, SALT lane). In the top strip of Figure 5, the support tree has wirelength 804,065 and exactly one violation: the root-to-sink path of t_2 has length 759,780, above its budget $B_{t_2} = 1.1125 \times 599,900 = 667,388.75$. SALT's repair (SALT lane, step 1) gives t_2 a private shortest wire of length 599,900. The other flows still need the old *trunk*—the long root-side run of wire that every flow shares—so only 126,540 of the old wire comes free, and wirelength jumps to 1,277,425. Postprocessing (step 2) then consolidates the tree onto the private wire the repair just added,

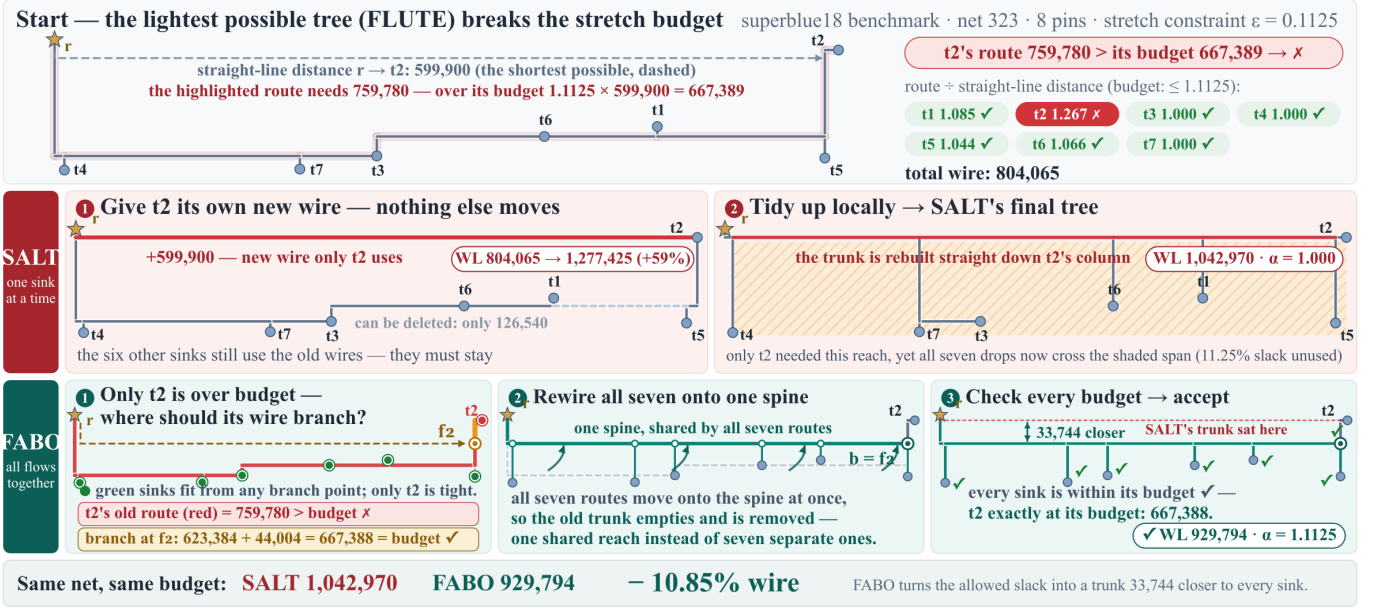


Fig. 5: **One recorded net, two repairs (superblue18 net 323, eight pins, $\epsilon = 0.1125$).** SALT gives the only violating sink t_2 a private wire and then rebuilds the trunk on t_2 's column, using none of the allowed 11.25% stretch. FABO instead places the shared trunk at f_2 , rewires all seven flows onto it, and verifies every budget. Under the same budgets, FABO reduces wirelength from 1,042,970 to 929,794 (10.85%).

rebuilding the trunk straight down t_2 's column: SALT's final tree has wirelength 1,042,970 at maximum stretch $\alpha = 1.000$.

The missed decision. SALT's final tree uses none of the permitted 11.25% stretch: the trunk is pinned to t_2 's column, and every *drop*—the wire from the trunk to one sink—crosses the full span between that column and its sink. The question SALT never asks is where the seven flows, as a group, should stop sharing one trunk. Reaching the shorter tree requires moving the trunk and every drop together. Per-sink repair never represents such a change, and SALT's postprocessing does not find it here: its local moves re-attach flows onto existing wire, and the deeper trunk does not yet exist. FABO asks exactly this question. In the FABO lane of Figure 5, step 2 rewires all seven flows onto a trunk 33,744 closer to the six other sinks; t_2 spends its permitted detour, while no other path gets longer. Step 3 verifies every budget and reports wirelength 929,794—a 10.85% reduction under the same constraints. In the terms of (6), FABO spends 67,488 more total path length—all of it t_2 's permitted detour—to save 180,664 more through sharing; the difference is exactly the 113,176 of wire removed. Figure 6 shows the same premature root-side splitting on eight high-degree nets; Section V shows that FABO saves more wire on higher-fanout nets and on nets where SALT shares less root-side wire.

B. Feasible Breakpoint Intervals

One interval per flow. Let the light support tree T^0 (FLUTE [3] in our implementation) be rooted at the root pin r , and let $T^0[u, v]$ denote its unique path between points u and v . Placing the breakpoint of sink t at a point $x \in T^0[r, t]$ means connecting r to x by a shortest rectilinear wire and

keeping the support suffix from x to t , as in Figure 4(b). The new root-to-sink route then has length

$$s_t(x) = d_{T^0}(x, t), \quad \phi_t(x) = d_M(r, x) + s_t(x). \quad (7)$$

A point x is *feasible* for sink t when $\phi_t(x) \leq B_t$. Here ϕ_t measures the route before any coincident wire merges; the realized tree path can only be shorter (Theorem 2), so feasibility under ϕ_t is a safe criterion. Along $T^0[r, t]$, *rootward* means closer to r and *deeper* means farther from r .

Moving x rootward lengthens the kept suffix by exactly the distance moved, while the Manhattan connection $d_M(r, x)$ can shrink by at most that distance—and may even grow—so ϕ_t never decreases rootward. The feasible points therefore form one suffix that starts at the rootward-most feasible point f_t :

$$I_t = \{x \in T^0[r, t] : \phi_t(x) \leq B_t\} = T^0[f_t, t]. \quad (8)$$

We call I_t the *feasible breakpoint interval* of sink t and of its flow. If the whole support path is feasible, then $f_t = r$. If f_t falls inside an edge, FABO inserts it as a vertex, changing neither geometry nor wirelength.

Lemma 1 (Feasible-interval property): For every sink t , the feasible set in (8) is a nonempty connected suffix of the support path $T^0[r, t]$.

Proof. Let x' be rootward of x on $T^0[r, t]$, and let $\Delta = d_{T^0}(x', x)$. Since x lies on $T^0[x', t]$, the suffix satisfies $s_t(x') = s_t(x) + \Delta$. The triangle inequality gives $d_M(r, x') \geq d_M(r, x) - \Delta$. Adding both, $\phi_t(x') \geq \phi_t(x)$: moving deeper never increases the path length. The sink itself is feasible because $\phi_t(t) = d_M(r, t) \leq B_t$, so the feasible set is exactly one nonempty suffix ending at t . $\square$

The lemma makes the interval exact for this criterion: I_t contains precisely the breakpoints whose guaranteed route

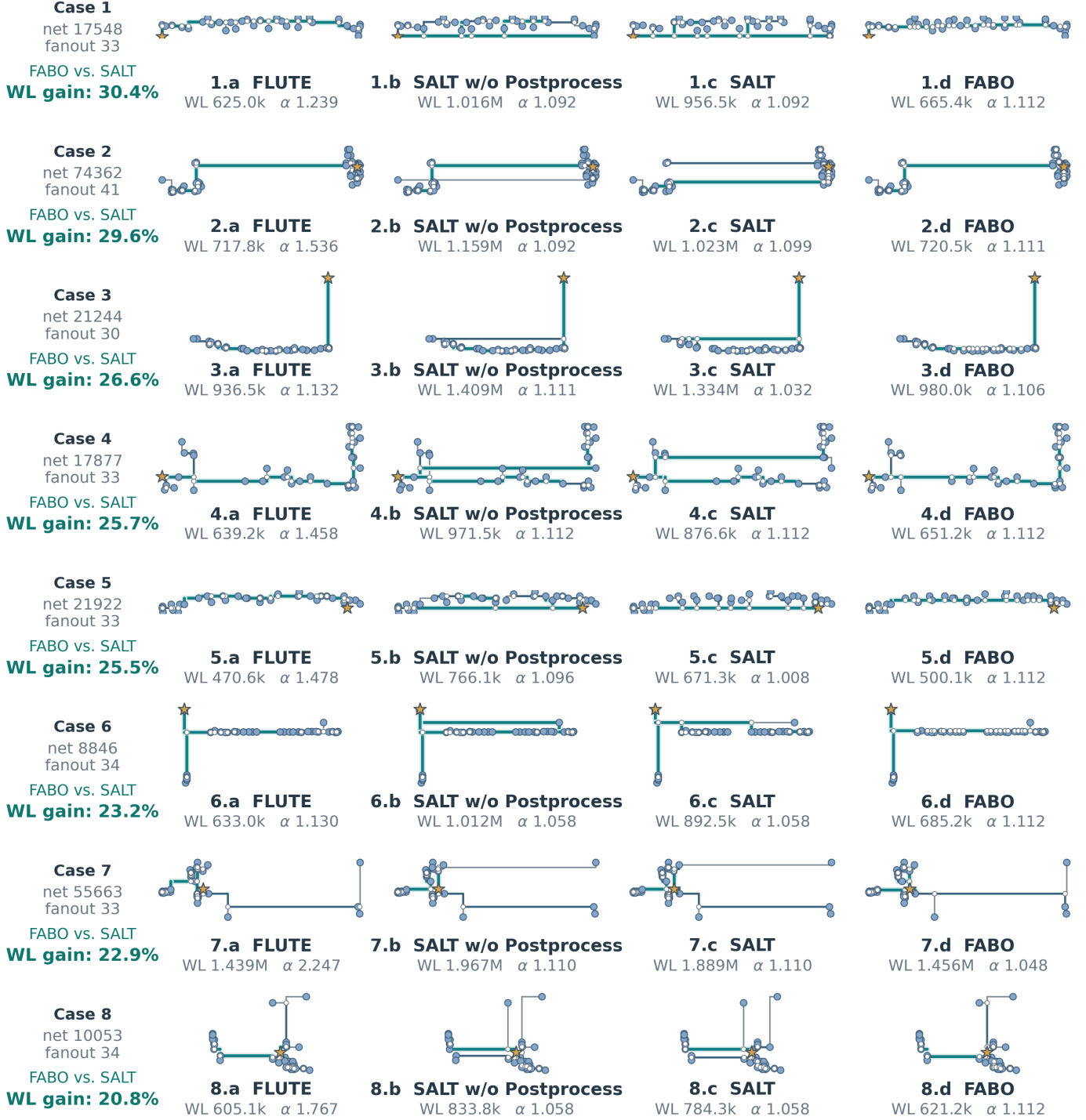


Fig. 6: **Eight high-degree examples of premature root-side splitting.** All cases use $\epsilon = 0.1125$. The four columns show the FLUTE support, SALT before postprocessing, postprocessed SALT, and FABO. The gold star marks the root pin r , filled blue circles mark sink pins, and hollow circles mark Steiner branch points. Edge color indicates how many root-to-sink paths share each edge. Panels in each row use the same plotting window and equal axis scale. Each gain is FABO's wirelength reduction relative to final SALT on that row.

meets the budget. The FABO lane of Figure 5, step 1, shows the resulting interval layout. Every flow except t_2 's is feasible at every point of its support path ($f_t = r$; green sinks). For t_2 , the boundary f_2 satisfies $\phi_{t_2}(f_2) = 623,384 + 44,004 = 667,388 \leq B_{t_2}$, leaving essentially no slack.

C. Shared Breakpoint Selection

Select for the group, not for one sink. A breakpoint a can serve sink t exactly when $a \in I_t$. If one point lies in several intervals, the corresponding flows can share one wire

from the root to that point and separate only afterward, as in Figure 4(c). FABO therefore selects a breakpoint set A that intersects every interval,

$$A \cap I_t \neq \emptyset, \quad \forall t \in P, \quad (9)$$

and assigns each sink the deepest selected point in its interval, keeping the suffix from that point:

$$a_t = \arg \max_{a \in A \cap I_t} \delta_0(a), \quad S_t = T^0[a_t, t], \quad (10)$$

where $\delta_0(x) = d_{T^0}(r, x)$ is the support depth of point x . The maximum in (10) is unique because all points of $A \cap I_t$ lie on the single path $T^0[r, t]$.

Feasible initialization. FABO initializes A greedily: it processes the *proper* intervals—those with $f_t \neq r$ —in non-increasing depth $\delta_0(f_t)$ and adds the boundary f_t whenever the current set misses I_t ; it then adds the root, which covers every remaining interval.

Theorem 1 (Minimum initial breakpoint set): The boundaries added by the greedy pass form a minimum-size set among all sets that intersect every interval with $f_t \neq r$.

Proof. Call the interval that caused a point to be added the point's *witness*; the greedy set has one point per witness. It suffices to show that witnesses are pairwise disjoint, because any valid set must place a distinct point inside each interval of a disjoint family. Suppose witnesses I_i and I_j share a point x , where I_i was processed first, so $\delta_0(f_i) \geq \delta_0(f_j)$. Because $x \in I_i$, the boundary f_i is rootward of x , hence $f_i \in T^0[r, x]$; likewise $f_j \in T^0[r, x]$. Points on one root path are totally ordered by depth, so f_i lies on $T^0[f_j, x]$, which is contained in I_j . Thus f_i covered I_j before I_j was processed, contradicting that I_j is a witness. $\square$

In the FABO lane of Figure 5, step 1 shows that only t_2 's interval excludes the root, so the greedy cover is $A = \{r, f_2\}$: sink t_2 is assigned $a_{t_2} = f_2$ and keeps its 44,004 suffix, while every other flow is assigned the root and keeps its complete support path. The cover thus involves no overlap decision; its contribution is placement. f_2 keeps the longest suffix allowed by t_2 's budget. On this net, the same point also places the shared trunk as close as possible to the other six sinks without violating that budget. Section IV-E shows how the other flows consolidate onto this wire; overlap drives the selection when several flows are tight at once, where one boundary covers many intervals and Theorem 1 keeps the count minimum.

Why selection does not stop here. The minimum cover is a guaranteed-feasible starting point, not the wirelength optimum: fewer breakpoints may shorten the new root connections, while deeper breakpoints may release more support wire. FABO resolves this trade-off with the flow-aware account introduced next, and decides acceptance only on complete trees (Section IV-E).

D. Flow-Aware Refinement

Count physical edges, not paths. FABO refines a breakpoint set A by proposing one deeper feasible point a at a time. Under $A \cup \{a\}$, every sink is reassigned by the deepest-point rule of (10):

$$a'_t(a) = \arg \max_{b \in (A \cup \{a\}) \cap I_t} \delta_0(b), \quad S'_t(a) = T^0[a'_t(a), t]. \quad (11)$$

The value of a proposal is the support wire it frees. Summing every flow's shortened suffix would count one shared edge many times, so FABO accounts per edge. For a retained support edge e , define its *edge-user set*

$$Q_e = \{t \in P : e \in S_t\} \quad (12)$$

and its *leaving-flow set* under proposal a ,

$$Y_e(a) = \{t \in P : e \in S_t \setminus S'_t(a)\}. \quad (13)$$

The edge is released exactly when every flow using it leaves (Figure 4(c)):

$$Y_e(a) = Q_e. \quad (14)$$

Lemma 2 (Exact release condition): After all flows are reassigned by (11), a currently retained support edge e belongs to no retained suffix if and only if (14) holds.

Proof. Reassignment only moves breakpoints deeper, so $S'_t(a) \subseteq S_t$ and no flow gains edges; every flow using e afterward already belongs to Q_e . Edge e survives exactly when some flow $t \in Q_e$ still uses it, that is, when $e \in S'_t(a)$ for some $t \in Q_e$ —equivalently, when $Y_e(a) \neq Q_e$. $\square$

The released support length is therefore

$$G(a) = \sum_{\substack{e: Q_e \neq \emptyset \\ Y_e(a) = Q_e}} \ell(e), \quad (15)$$

where $\ell(e)$ is the length of support edge e .

Figure 5 shows why released wire must be counted per edge. After SALT step 1, most of the old trunk remains because the other six flows still use it; only 126,540 of the 804,065 support wire becomes unused. After FABO step 2 moves all seven flows to the new spine, the old trunk has no users and can be removed entirely. In this example, standard postprocessing performs the migration; the edge-level gain defined above is used to rank candidate breakpoints during refinement. The gain $G(a)$ only ranks proposals; acceptance is decided on the complete tree, where the new root connections are also paid.

E. Complete-Tree Construction and Acceptance

Evaluate the object that will be returned. For a breakpoint set A , FABO builds a rectilinear *connector* $C(A)$ that spans the root and the selected breakpoints and reaches every breakpoint at its Manhattan distance from the root:

$$d_{C(A)}(r, a) = d_M(r, a), \quad \forall a \in A. \quad (16)$$

Grafting every retained suffix S_t under its breakpoint a_t and removing redundant Steiner nodes yields the raw tree $T_{\text{raw}}(A)$.

Theorem 2 (Feasibility of a realized candidate): If every sink t is assigned a breakpoint $a_t \in I_t$, the connector satisfies (16), and realization preserves every retained suffix and returns a tree, then $T_{\text{raw}}(A)$ satisfies every budget in (5).

Proof. For each sink t , the realized tree contains the connector path from r to a_t followed by the suffix S_t , and the tree path from r to t is no longer than this walk:

$$d_{T_{\text{raw}}(A)}(r, t) \leq d_M(r, a_t) + s_t(a_t) = \phi_t(a_t) \leq B_t, \quad (17)$$

where the last step uses $a_t \in I_t$. $\square$

Realization and postprocessing. $\text{FULLREALIZE}(A)$, used on finalist candidates, builds the connector with SALT’s rectilinear Steiner arborescence construction [4]; $\text{FASTREALIZE}(A)$, used while scanning proposals, builds it by merging overlapping shortest root paths. Both constructions satisfy (16); suffix preservation and tree validity are enforced by construction and re-checked by the evaluator, and merging coincident wire only shortens root-to-sink paths, so the bound of Theorem 2 survives. Each realized tree is additionally passed through SALT’s fixed postprocessing, which may re-attach a flow onto the connector when doing so shortens the tree without violating any budget; the raw and postprocessed forms are both evaluated, and FULLREALIZE and FASTREALIZE return the better valid form. Each of these local moves relocates one attachment, so the passes do not assemble a long new trunk on their own—a coordinated migration would have to cross non-improving intermediate trees—but they readily exploit a trunk that exists: moving a budget-loose drop onto it is a single improving legal move. The same postprocessing therefore stalls on SALT’s tree and completes the consolidation on FABO’s.

Acceptance. The evaluator requires a connected, acyclic rooted tree that contains every pin, checks every root-to-sink budget, and measures exact wirelength. Among valid trees, lower wirelength wins, and maximum stretch α breaks a wirelength tie; we write this order as $\prec_{\text{lex}}$. The FABO lane of Figure 5 separates these roles: step 1 selects f_2 and builds its shortest root connector, while step 2 moves the other six flows onto the resulting spine, leaving them on shortest root paths and deleting the old trunk. FABO step 3 of Figure 5 shows the accepted result: every budget holds, the path of t_2 is 667,388 against its budget of 667,388.75, and wirelength is 929,794 against SALT’s 1,042,970 under the same constraints.

F. FABO Candidate Search

Two-stage search. Algorithm 1 summarizes FABO on one support tree. Stage one computes the intervals, builds the minimum cover, and realizes it; Theorem 2 guarantees its budgets and the evaluator confirms them, so this stage already yields a valid tree. Stage two proposes a bounded number of deeper breakpoints, ranks them by a score dominated by the flow-aware gain, commits a proposal only when its fast-realized tree is valid and shorter, and finally re-realizes the enlarged set in full.

Search scope. Every bound in the search is a fixed constant, identical for all nets and tolerances and released with the code: the pool keeps a constant number of candidates, each round fast-realizes only the top-ranked proposals, and a round with little gain ends the search early. The ranking is a fixed weighted score dominated by the released length $G(a)$, plus two lighter terms: one rewards the total depth by which reassigned flows move, and one discounts proposals far from every already selected breakpoint—a proxy for the connector wire the addition will need. On nets with at least 31 pins, FABO repeats the whole search on a second, higher-accuracy FLUTE support tree and returns the better valid result.

Algorithm 1 FABO on one light support tree.

Require: Root r , sink set P , path budgets B_t , support tree T^0

Ensure: A feasible rectilinear routing tree

- 1: Compute f_t and $I_t = T^0[f_t, t]$ for every sink $t \in P$
- 2: $A \leftarrow \emptyset$
- 3: **for** each interval with $f_t \neq r$, in nonincreasing $\delta_0(f_t)$ **do**
- 4: **if** $A \cap I_t = \emptyset$ **then**
- 5: $A \leftarrow A \cup \{f_t\}$
- 6: **end if**
- 7: **end for**
- 8: $A \leftarrow A \cup \{r\}$
- 9: $T_\star \leftarrow \text{FULLREALIZE}(A)$; $T_{\text{scan}} \leftarrow \text{FASTREALIZE}(A)$
- 10: Form a bounded pool Q of deeper points from the feasible intervals
- 11: **for** a bounded number of refinement rounds **do**
- 12: Rank every $a \in Q \setminus A$ by a fixed score dominated by $G(a)$
- 13: Fast-realize and validate the leading proposals $A \cup \{a\}$
- 14: **if** no valid proposal improves T_{scan} **then**
- 15: **break**
- 16: **end if**
- 17: Commit the best proposal $a^\star$: $A \leftarrow A \cup \{a^\star\}$; update T_{scan}
- 18: **end for**
- 19: $T' \leftarrow \text{FULLREALIZE}(A)$
- 20: **if** $T' \prec_{\text{lex}} T_\star$ **then**
- 21: $T_\star \leftarrow T'$
- 22: **end if**
- 23: **return** $T_\star$

G. Guarantees and Scope

What is guaranteed. Lemma 1 gives exact feasible intervals; Theorem 1 minimizes only the number of initial non-root breakpoints; Theorem 2 proves budget feasibility if the connector and realization satisfy their stated conditions; and Lemma 2 makes edge-release accounting exact. These are properties of the construction, not the final acceptance guarantee. In the implementation, a fixed evaluator rejects any incomplete or budget-violating candidate. The validated stage-one tree initializes the incumbent, which is replaced only by a valid candidate that improves (WL, α) under $\prec_{\text{lex}}$. Thus, the evaluator and incumbent-update rule ensure that FABO returns a feasible tree no worse than its validated stage-one tree.

What remains heuristic. The minimum cover minimizes only the number of initial non-root breakpoints; it does not guarantee minimum final wirelength. Likewise, the gain $G(a)$ measures released support wire, not the net wirelength change after adding the connector. Candidate ranking, connector construction, support-tree choice, and postprocessing are therefore heuristics; complete-tree evaluation is the only acceptance rule.

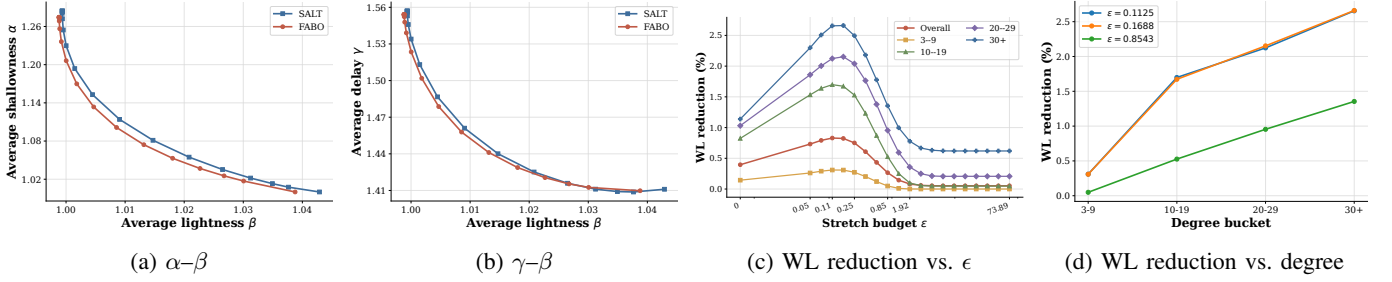


Fig. 7: **FABO dominates SALT across all 20 stretch tolerances.** Panels (a) and (b) show full-benchmark means at each ϵ ; FABO lies below and to the left of SALT in both plots. Panel (c) reports FABO’s full-benchmark wirelength reduction at each ϵ . Panel (d) reports the reduction within each pin-count range at the three shown tolerances.

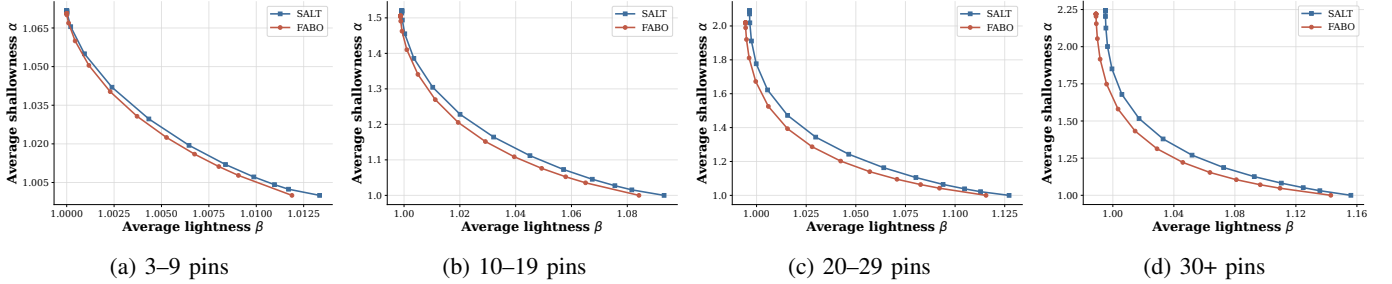


Fig. 8: **FABO dominates SALT in every pin-count range, with a wider gap at higher fanout.** Each point averages α and β over all nets in the labeled pin-count range for one method and one ϵ .

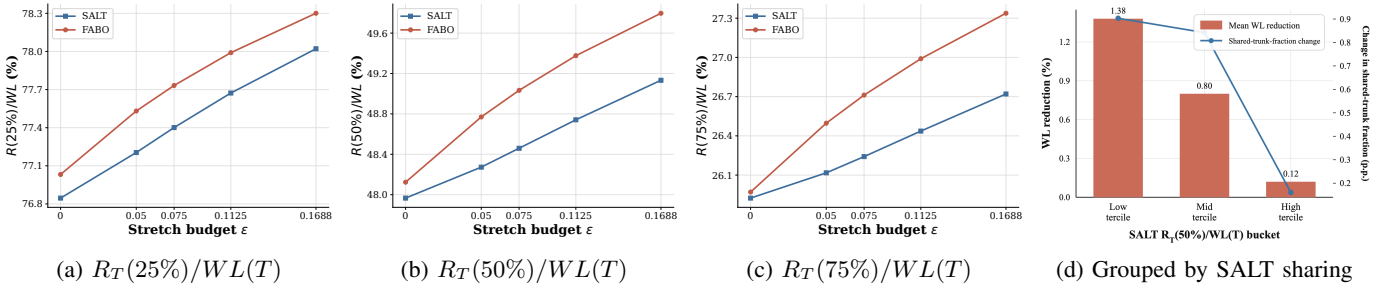


Fig. 9: **FABO keeps a larger fraction of wirelength on shared root-side trunks.** The metric $R_T(\rho)/WL(T)$ is the fraction of total wirelength in the root-connected trunk used by at least ρ of the net’s root-to-sink paths. Panels (a)–(c) report the mean sharing metric over all 1.29 million nets at the five shown tolerances. Panel (d) groups nets into terciles by SALT’s $R_T(50\%)/WL(T)$ at $\epsilon = 0.1125$. The bars show FABO’s mean wirelength reduction relative to SALT; the line shows FABO’s mean shared-trunk fraction minus SALT’s.

V. RESULTS

A. Experimental Setup

Benchmark and baseline. We evaluate FABO on the eight ICCAD15 Contest Problem B designs distributed by the official SALT repository flow [4], [22]: superblue1, superblue3, superblue4, superblue5, superblue7, superblue10, superblue16, and superblue18.¹ The benchmark contains 1,294,541 nets with at least three pins. We use

¹The released benchmark differs from the counts reported in the SALT paper. It contains no 2-pin nets and 1.29M nets with at least three pins, whereas [4] reports 2.38M nontrivial nets. Most of the difference is in the 3–9-pin bucket. This discrepancy does not affect our comparison: FABO and the official SALT implementation use the same released files and the same evaluator.

postprocessed SALT as the direct baseline because it targets the same objective: minimum wirelength under the same per-sink path-length limit. For every net and tolerance, FABO and SALT receive the same input and are evaluated by the same checker. Because 3–9-pin nets make up 69.9% of the benchmark, while only 58.3k nets have at least 30 pins, we report both full-benchmark and fanout-stratified results (Table I).

Metrics. The stretch tolerance ϵ limits every root-to-sink path to $(1 + \epsilon)$ times its Manhattan distance. We use three quality metrics. For each sink, path stretch is its root-to-sink path length divided by its Manhattan distance; shallowness α is the maximum of these ratios. FLUTE-normalized wirelength $\beta = WL(T)/WL(T_{\text{FLUTE}})$ is total tree wirelength divided by the FLUTE wirelength for the same net. Delay γ is the

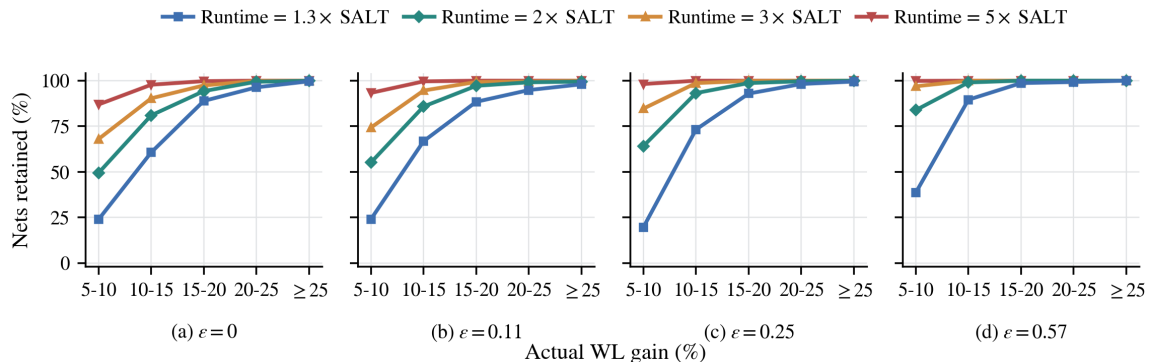


Fig. 10: **FABO-FAST recovers the high-gain nets first.** The x-axis bins nets by the wirelength reduction obtained when FABO runs on every net. The y-axis is the fraction of each bin selected by FABO-FAST. Each curve represents a total runtime budget normalized by the all-net SALT runtime; the budget includes prediction and FABO calls on selected nets. The four panels use $\epsilon = 0, 0.1125, 0.253125, \text{ and } 0.56953125$, with a separate leave-one-design-out predictor trained for each ϵ .

TABLE I: **Benchmark size and fanout distribution.** Net degree is the number of pins; each entry counts nets directly from the eight ICCAD15 .nets files.

Design	3–9 ($\times 10^3$)	10–19 ($\times 10^3$)	20–29 ($\times 10^3$)	30–39 ($\times 10^3$)	≥ 40 ($\times 10^3$)	≥ 3 ($\times 10^3$)
superblue1	135	23.3	11.2	5.8	0.9	176.2
superblue10	147.7	31.3	13.8	9.5	1.2	203.5
superblue16	114	17.2	7.3	5.3	0.3	144.2
superblue18	54.5	24.5	10.9	5.1	0.6	95.5
superblue3	102.1	35	15.4	6.1	1.1	159.8
superblue4	74.2	16.9	8.7	3.8	0.5	104
superblue5	123.2	18.2	7.6	4.7	0.7	154.3
superblue7	154.4	62.5	27.5	11	1.7	257.1
Total	904.9	228.8	102.5	51.3	7	1294.5

normalized maximum Elmore-delay metric used by SALT. Lower is better for all three. At fixed ϵ , both algorithms satisfy the same per-sink limit, so lower mean wirelength means lower routing cost under the same feasibility constraint.

B. FABO Dominates SALT at Every Constraint

Main result. At each of the 20 stretch-tolerance settings, FABO has lower mean wirelength than postprocessed SALT. The largest full-benchmark reduction is 0.83% at $\epsilon = 0.1125$: mean FLUTE-normalized wirelength is 1.0312 for SALT and 1.0227 for FABO.

Shape of the sweep. At the strictest constraint, $\epsilon = 0$, both methods reach $\alpha = 1$. SALT’s β is 1.0429, whereas FABO’s is 1.0388. The reduction peaks at 0.83% once the constraint allows enough slack to move shared breakpoints jointly across several paths. At larger ϵ , both methods approach the low-wirelength FLUTE topology, so FABO’s advantage narrows but remains positive.

Pareto dominance. At every ϵ , FABO has lower mean β without increasing mean α or γ ; FABO therefore dominates SALT at all 20 operating points in both trade-off plots. In the sampled sweep, FABO first reaches $\beta < 1.001$ at $\alpha = 1.2062$, whereas SALT first reaches it at $\alpha = 1.2298$. On the delay frontier, FABO first reaches $\beta < 1.001$ at $\gamma = 1.5235$, whereas SALT first reaches it at $\gamma = 1.5340$.

C. The Gain Grows with Fanout

Fanout trend. FABO dominates SALT in all four pin-count ranges, and its advantage grows with net degree. For each ϵ , we report FABO’s percentage reduction in mean FLUTE-normalized wirelength relative to SALT. Across the 20 tolerances, the maximum reduction is 0.31% for 3–9 pins, 1.70% for 10–19, 2.15% for 20–29, and 2.66% for at least 30 pins. At each of the three tolerances shown in Figure 7(d), the reduction increases across the four pin-count ranges. Because 3–9-pin nets make up 69.9% of the benchmark, they dominate the full-benchmark average; this is why the overall peak of 0.83% is much smaller than the high-fanout peaks. Higher-fanout nets have more root-side edges used by several sink paths. SALT repairs each sink path separately, which can duplicate this shared wire; FABO instead moves shared breakpoints jointly across paths. The next subsection tests whether preserving shared root-side wire explains FABO’s larger gains on high-fanout nets.

D. Why FABO Saves Wire: A Larger Shared-Trunk Fraction

Mechanism test. If FABO’s gain comes from preserving shared root-side wire, its trees should keep a larger fraction of their wirelength on heavily shared trunks. For a threshold ρ , consider the root-connected trunk formed by edges used by at least ρ of the net’s root-to-sink paths. We denote its length by $R_T(\rho)$ and use $R_T(\rho)/WL(T)$ as the shared-trunk fraction. We compute the sharing metric over all 1.29 million nets at the five shown tolerances.

For every shown pair of ϵ and ρ , FABO’s mean shared-trunk fraction is higher than SALT’s. At $\epsilon = 0.1125$ and $\rho = 50\%$, SALT’s mean shared-trunk fraction is 48.74%, whereas FABO’s is 49.38%. Panel (d) divides nets into terciles using SALT’s $R_T(50\%)/WL(T)$. Relative to SALT, FABO reduces mean wirelength by 1.38% in the low-sharing tercile, 0.80% in the middle tercile, and 0.12% in the high-sharing tercile. The largest reduction therefore occurs on nets for which SALT preserves the least shared wire. Together, the fanout and trunk results support the proposed mechanism: FABO’s trees allocate

a larger fraction of their wirelength to shared root-side trunks, especially on the nets where SALT shares the least.

E. FABO-FAST: Budget-Controlled FABO

Why FABO-FAST. Running FABO on every net gives the lowest mean wirelength, but at $\epsilon = 0.1125$ the complete flow takes $10.66\times$ the all-net SALT runtime. FABO-FAST first runs SALT on every net. A learned predictor ranks nets by their expected FABO gain, and FABO runs in that order until the total runtime reaches the budget. An unselected net retains its SALT tree. For a selected net, the final output is the shorter feasible tree produced by SALT and FABO.

Gain and recovery. For net i , let S_i and F_i be the wirelengths of its feasible SALT and FABO trees. Because the final output uses the shorter tree, we define the realized percentage reduction as

$$g_i = 100 \frac{\max(S_i - F_i, 0)}{S_i}. \quad (18)$$

For a gain interval I , such as $[20\%, 25\%)$, let $\mathcal{A}$ be the nets selected by FABO-FAST. We define recovery as the fraction of all nets whose full-FABO gain lies in I that FABO-FAST selects:

$$Q_I(\mathcal{A}) = 100 \frac{|\{i : i \in \mathcal{A}, g_i \in I\}|}{|\{i : g_i \in I\}|}. \quad (19)$$

The denominator is computed by running FABO on every net at the same ϵ . Thus, $Q_I = 100\%$ means FABO-FAST selects every net whose full-FABO gain falls in interval I .

Predictor. The depth-4, 96-tree XGBoost predictor takes 22 features computed before FABO runs. These features describe pin count, bounding-box geometry, FLUTE wirelength and path statistics, and SALT wirelength, stretch, and delay. We use eight-fold leave-one-design-out evaluation. Each fold trains on seven designs and predicts all nets in the held-out design, so every net receives exactly one out-of-design prediction. We train a separate predictor for each plotted ϵ .

Runtime accounting. Let T_{SALT} be the time to run SALT on every net. The FABO-FAST runtime T_{FAST} includes T_{SALT} , one batch prediction, and FABO calls on selected nets. We report the budget multiplier $b = T_{\text{FAST}}/T_{\text{SALT}}$. At $\epsilon = 0.1125$, SALT takes 62.43 s, running FABO on all nets adds 602.83 s, and batch prediction takes 0.872 s.

Runtime-quality trade-off. At $b = 1.3$, across the four plotted ϵ values, FABO-FAST recovers 99.0%–100% of nets with at least 25% gain and 96.0%–99.3% of nets with 20%–25% gain. Recovery is lower for smaller gains: 88.5%–99.2% for the 15%–20% group and 21.0%–45.3% for the 5%–10% group. The monotonic drop in recovery as gain decreases shows that the predictor ranks the highest-gain nets first. At $b = 2$, recovery is 98.9%–100% for both the 20%–25% group and the at-least-25% group. As b increases, FABO-FAST selects more nets and approaches the result of running FABO on every net.

Why high-gain nets are predictable. A large increase in wirelength from FLUTE to SALT, or a large change in path stretch, suggests that SALT made an expensive structural repair to satisfy the path constraint. Such nets give FABO's

coordinated breakpoints the most opportunity to reduce wirelength.

VI. CONCLUSION

We developed an agent-guided framework that combines parallel language-model exploration with independent checking to discover and validate routing-tree algorithms. Applied to SALT, it discovered that sink-by-sink repair cannot coordinate breakpoints across paths sharing root-side wire, causing premature splitting and duplicated wire. This finding led to FABO, which represents each path's feasible breakpoint choices as an interval and jointly selects shared breakpoints while preserving every sink's stretch budget. Across 1.29 million nets, FABO reduces average FLUTE-normalized wirelength at every setting, with peak reductions of 0.83% overall and 2.66% for nets with at least 30 pins; its advantage grows with fanout, and its trees retain more shared root-side wire. FABO-FAST makes this improvement practical by identifying and optimizing most high-gain nets within $1.3\times$ SALT's runtime.

REFERENCES

- [1] C. J. Alpert, A. B. Kahng, C. C. N. Sze, and Q. Wang, "Timing-driven steiner trees are (practically) free," in *Proceedings of the 43rd ACM/IEEE Design Automation Conference*, 2006, pp. 389–392.
- [2] C. C. N. Chu and Y.-C. Wong, "Flute: Fast lookup table based rectilinear steiner minimal tree algorithm for vlsi design," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 27, no. 1, pp. 70–83, 2008.
- [3] C. Chu, "Flute: Fast lookup table based wirelength estimation technique," in *IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, 2004, pp. 696–701.
- [4] G. Chen, P. Tu, and E. F. Y. Young, "Salt: Provably good routing topology by a novel steiner shallow-light tree algorithm," in *2017 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2017, pp. 569–576.
- [5] C. J. Alpert, W.-K. Chow, K. Han, A. B. Kahng, Z. Li, D. Liu, and S. Venkatesh, "Prim-dijkstra revisited: Achieving superior timing-driven routing trees," in *Proceedings of the ACM/IEEE International Symposium on Physical Design (ISPD)*, 2018, pp. 10–17.
- [6] C. J. Alpert, T. C. Hu, D. J.-H. Huang, A. B. Kahng, and D. R. Karger, "Prim-dijkstra tradeoffs for improved performance-driven routing tree design," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 14, no. 7, pp. 890–896, 1995.
- [7] Z. Chen, H. Yao, and X. Yin, "Patlabor: Pareto optimization of timing-driven routing trees," in *Proceedings of the 62nd ACM/IEEE Design Automation Conference*, 2025, pp. 1–7.
- [8] H. Wu, X. Li, L. Chen, B. Yu, and W. Zhu, "Delay-driven rectilinear steiner tree construction," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 44, no. 5, pp. 1928–1941, 2025.
- [9] B. Romera-Paredes, M. Barekatin, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, P. Kohli, and A. Fawzi, "Mathematical discoveries from program search with large language models," *Nature*, vol. 625, pp. 468–475, 2024.
- [10] F. Liu, X. Tong, M. Yuan, X. Lin, F. Luo, Z. Wang, Z. Lu, and Q. Zhang, "Evolution of heuristics: Towards efficient automatic algorithm design using large language model," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, 2024, pp. 32 201–32 223.
- [11] A. Novikov *et al.*, "Alphaevolve: A coding agent for scientific and algorithmic discovery," 2025.
- [12] C. Yu, R. Liang, C.-T. Ho, and H. Ren, "Autonomous code evolution meets np-completeness," *arXiv preprint arXiv:2509.07367*, 2025.
- [13] Z. He, H. Wu, X. Zhang, X. Yao, S. Zheng, H. Zheng, and B. Yu, "Chatada: A large language model powered autonomous agent for eda," 2023.
- [14] A. Ghose, A. B. Kahng, S. Kundu, and Z. Wang, "Orfs-agent: Tool-using agents for chip design optimization," in *Proceedings of the 2025*

ACM/IEEE International Symposium on Machine Learning for CAD, 2025, pp. 1–13.

- [15] G. Pasandi, K. Kunal, V. Tej, K. Shan, H. Sun, S. Jain, C. Li, C. Deng, T.-D. Ene, H. Ren, and S. Pratty, “Jarvis: A multi-agent code assistant for high-quality eda script generation,” 2025.
- [16] F. Hutter, H. H. Hoos, K. Leyton-Brown, and T. Stützle, “Paramils: An automatic algorithm configuration framework,” *Journal of Artificial Intelligence Research*, vol. 36, pp. 267–306, 2009.
- [17] F. Hutter, H. H. Hoos, and K. Leyton-Brown, *Sequential Model-Based Optimization for General Algorithm Configuration*. Springer, 2011, pp. 507–523.
- [18] M. López-Ibáñez, J. Dubois-Lacoste, L. Pérez Cáceres, M. Birattari, and T. Stützle, “The irace package: Iterated racing for automatic algorithm configuration,” *Operations Research Perspectives*, vol. 3, pp. 43–58, 2016.
- [19] D. Fernández-Baca and L. Liu, “Testing the agreement of trees with internal labels,” *Algorithms for Molecular Biology*, vol. 16, no. 1, p. 22, 2021.
- [20] M. Hendriksen and A. Francis, “A partial order and cluster-similarity metric on rooted phylogenetic trees,” *Journal of Mathematical Biology*, vol. 80, no. 5, pp. 1265–1290, 2020.
- [21] M. Segal, “On piercing sets of axis-parallel rectangles and rings,” in *Algorithms—ESA ’97*, ser. Lecture Notes in Computer Science, vol. 1284. Springer, 1997, pp. 430–442.
- [22] M.-C. Kim, J. Hu, J. Li, and N. Viswanathan, “Iccad-2015 cad contest in incremental timing-driven placement and benchmark suite,” in *2015 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2015, pp. 921–926.



Shang Liu received the B.E. degree in Automation Science and Electrical Engineering from Beihang University, Beijing, China, in 2023. He is currently pursuing the Ph.D. degree with the Department of Electronic and Computer Engineering, Hong Kong University of Science and Technology, Hong Kong. His research interests include agile VLSI design methodologies and Artificial Intelligence.



Wenji Fang is currently a Ph.D. student with the Department of Electronic and Computer Engineering at the Hong Kong University of Science and Technology. He received his M.Phil. degree in Microelectronics from the Hong Kong University of Science and Technology (Guangzhou) in 2024, and his B.Eng. degree from Nanjing University of Aeronautics and Astronautics in 2021. His research interests include Electronic Design Automation (EDA) and VLSI design verification.



Jing Wang received the B.S. degree in Electrical Information Engineering from Peking University, Beijing, China, in 2022, and the master’s degree in Artificial Intelligence from the Department of Statistics and Actuarial Science, The University of Hong Kong, China, in 2023. He is currently pursuing the Ph.D. degree with the Department of Electronic and Computer Engineering, Hong Kong University of Science and Technology, Hong Kong. His research interests include agile VLSI design methodologies and Artificial Intelligence.



timing analysis.

Hongxin Kong received the B.S. degree in electrical engineering from Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2016, and the Ph.D. degree in computer engineering from Texas A&M University, College Station, TX, USA, in 2020. He is currently a senior staff R&D engineer with Synopsys, Inc. Prior to joining Synopsys, he was with Advanced Micro Devices (AMD) and Cadence. His research interests include electronic design automation (EDA), GPU-accelerated physical design algorithms, place and route (P&R), and static



Yao Lu received the B.E. degree from the School of Electronic Science and Engineering, Southeast University, Nanjing, China, in 2020, and the master’s degree from the School of Microelectronics, Fudan University, Shanghai, China, in 2023. She is currently pursuing the Ph.D. degree with the Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, Hong Kong. Her current research interests focus on machine learning applications in EDA.



Zhiyao Xie is an Assistant Professor in the Department of Electronic and Computer Engineering (ECE) at the Hong Kong University of Science and Technology (HKUST). He received his Ph.D. degree from Duke University in 2022 and the B.Eng. degree from City University of Hong Kong in 2017. His research focuses on AI-driven techniques for EDA and VLSI design. He has received multiple prestigious awards, including the ACM SIGDA Outstanding New Faculty Award 2026, ACM ASPLOS 2026 Best Paper Award, IEEE/ACM MICRO 2021 Best Paper Award, ASP-DAC 2023 Best Paper Award, ACM SIGDA SRF Best Research Poster Award 2022, ACM Outstanding Ph.D. Dissertation Award in EDA 2023, EDAA Outstanding Ph.D. Dissertation Award 2023, and Hong Kong Research Grants Council (RGC) Early Career Award 2023.